



课程项目开发规范

Created 2026年5月9日 21:07

Linux系统管理与Shell编程

课程项目开发规范（本科教学版）

适用课程：《Linux系统管理与Shell编程》

参考教材：How Linux Works

一、课程项目目标

本课程项目旨在让学生：

- 学会 Linux 环境下的软件开发与系统管理
- 掌握 Shell 编程与自动化处理
- 学会利用 Linux 命令解决实际问题
- 初步理解团队协作与项目开发流程
- 能够利用 AI/Agent 工具提高开发效率

本项目强调：

“能运行、能解释、能测试、能维护”

而不仅仅是“代码能写出来”。

二、项目组队要求

1. 组队人数

每组 3~5 人。

建议：

- 3人：基础功能项目
 - 4~5人：较复杂项目
-

2. 分工要求

每位成员必须承担明确任务。

建议包括：

- Linux环境与系统配置
- Shell脚本开发
- 功能实现
- 测试与调试
- 文档与演示

项目报告中必须说明：

- 每个人负责的内容
 - 实际完成的工作
-

三、项目方向要求

项目必须是：

Linux 系统下的实际应用开发或系统管理工具。

允许：

- Shell工具
- 自动化脚本
- 系统监控
- 日志分析
- Linux运维工具
- Web辅助管理工具
- AI Agent Skills

- 数据处理工具

不限制编程语言，但：

必须体现 Linux 系统管理与 Shell 编程相关内容。

四、项目最低技术要求（重要）

项目必须至少使用以下内容中的 **4项**：

- Shell脚本（bash）
 - Linux系统命令
 - grep/sed/awk 文本处理
 - 管道与重定向
 - 文件与权限管理
 - 进程管理
 - 日志分析
 - crontab 定时任务
 - SSH/scp 远程管理
 - 网络相关命令
 - Python 调用 Linux 功能
 - Linux 服务管理
 - Git 代码管理
 - Docker（加分项）
 - 数据库（SQLite/MySQL/PostgreSQL）
 - Agent 工具调用（如AI Agent Skills）
-

五、推荐项目类型

1. Linux系统管理类

例如：

- 系统监控工具
 - CPU/内存/磁盘监控
 - 自动备份工具
 - SSH批量管理工具
 - 日志分析系统
-

2. 数据处理类

例如：

- Linux日志统计
 - 文本分析工具
 - 文件自动整理工具
 - 网站信息采集工具
-

3. 自动化工具类

例如：

- 自动部署脚本
 - 自动测试脚本
 - 自动巡检系统
 - 定时报告生成工具
-

4. AI Agent Skills（新增推荐）

由于 AI Agent 与自动化工具正在快速发展，允许学生开发：

- Linux 运维 Agent Skills
- 日志分析 Agent
- Shell 指令自动生成工具
- Linux 故障诊断 Agent
- 自动化巡检 Agent
- 多命令组合执行工具

项目重点不在“大模型训练”，而在：

- Linux工具调用
- Shell命令组合
- 自动化流程设计
- Agent执行逻辑

例如：

用户输入：

“检查磁盘空间问题”

系统自动：

- 执行 df
- 分析日志
- 输出诊断结果

这是非常适合作为课程项目的方向。

六、不建议的项目

以下项目不建议作为本课程项目：

- 纯网页前端页面
 - 纯GUI程序
 - 与Linux系统无关的项目
 - 游戏开发
 - 单纯调用AI聊天接口
 - 直接照搬网络教程且无明显修改
-

七、项目等级说明（重要）

考虑到本课程同时包含：

- 低年级学生（Linux基础较弱、缺少开发经验）
- 高年级学生（已有一定开发经验）

因此：

项目评分采用：

“分层评价机制”

即：

- 不同基础学生可以选择不同难度方向
- 更强调“是否真正完成并掌握”
- 避免仅以代码复杂度评分

课程鼓励：

“基础同学完成一个真正可运行的小项目”
“高年级同学完成一个具有工程性的Linux系统项目”

（一）低年级学生项目等级参考

适用于：

- Linux基础较弱
- 第一次接触项目开发
- 缺少软件工程经验
- 低年级本科生

重点考察：

- 是否真正掌握Linux基本操作
- 是否能够完成自动化任务
- 是否能够独立运行与测试

1. 基础项目（60~75分）

特点：

- 能完成基本功能
- 能在Linux环境运行
- 使用Shell脚本

- 使用基本Linux命令
- 有简单文档

例如：

- 文件自动备份脚本
- CPU/内存监控脚本
- 自动整理文件工具
- 简单日志统计工具

要求：

- 能正常运行
- 有基本输入输出
- 有简单测试

适合：

- Linux初学者
 - 基础薄弱学生
-

2. 良好项目（75~90分）

特点：

- 有多个功能模块
- 有较好的自动化能力
- 使用多种Linux技术
- 有测试与错误处理

例如：

- 自动巡检工具
- SSH批量执行脚本
- Web日志分析工具
- 自动化部署脚本

要求：

- 有较完整功能

- 有一定代码结构
 - 有测试案例
 - 有较清晰文档
-

3. 优秀项目（90分以上）

特点：

- 能解决实际问题
- 自动化程度较高
- 有较好的工程结构
- 有一定创新性

例如：

- Linux监控平台
- 自动告警系统
- 日志分析与可视化工具
- 简单Agent自动运维工具

优秀项目通常：

- 有较完整系统设计
 - 有较好的可维护性
 - 能体现团队协作
-

（二）高年级学生项目等级参考

适用于：

- 有开发经验
- 学过软件工程相关课程
- 有一定项目基础
- 高年级本科生

重点考察：

- 工程能力
- Linux系统能力
- 自动化与架构能力
- AI/Agent工具使用能力

评分标准会更加严格。

1. 基础项目（60~75分）

特点：

- 能完成基本系统功能
- 使用Linux相关技术
- 有基本模块划分

例如：

- Linux系统监控工具
- 自动部署系统
- Shell+Python运维工具

对于高年级学生：

仅完成简单脚本功能：

❗ 难以获得高分。

2. 良好项目（75~90分）

特点：

- 有完整系统流程
- 有模块化设计
- 有较好的自动化能力
- 有较规范工程结构
- 使用Git进行持续开发

例如：

- SSH批量运维平台

- 自动巡检与日志分析系统
- Docker自动部署工具
- Web管理后台 + Linux脚本协同系统

要求：

- 有完整运行流程
 - 有错误处理
 - 有测试案例
 - 有较好的文档
-

3. 优秀项目（90分以上）

特点：

- 具有较强工程性
- 有完整自动化流程
- 有明显Linux系统特色
- 有较强创新性
- 能体现真实工程思维

例如：

- Linux运维Agent系统
- AI辅助故障诊断工具
- 多机器监控与告警平台
- 基于Agent Skills的自动巡检系统
- Docker容器自动编排工具

优秀项目通常：

- 有清晰系统架构
- 有完整开发过程
- 有较规范代码管理
- 有测试与性能分析
- 有较好的扩展能力

（三）关于“项目难度”的说明

课程评分：

| 不仅看“代码复杂度”

更看：

- 是否真正完成
- 是否真正理解
- 是否体现Linux课程内容
- 是否有完整开发过程

因此：

一个：

- 结构清晰
- 自动化完整
- 文档规范
- 测试充分

的小项目，

通常比：

- 功能很多
- 但无法运行
- 无法解释
- 没有开发过程

的大项目更容易获得高分。

（四）课程建议

低年级同学建议：

| “先完成一个稳定的小项目”

高年级同学建议：

“尝试更完整的Linux工程系统”

课程鼓励：

- 从实际问题出发
- 重视自动化
- 重视系统稳定性
- 重视工程实践能力

而不是单纯追求：

- 界面复杂
 - 功能数量
 - AI噱头。
-

八、项目开发流程

本课程采用：

“轻量化项目开发流程”

不要求复杂的软件工程文档。

九、项目时间安排

第1阶段：开题（第1周）

需要提交：

项目规划书（简化版）

内容不超过3页，包括：

1. 项目名称
2. 项目目标
3. 项目功能
4. 使用技术

5. 小组分工

6. 开发计划

7. 可能风险

每组进行：

- 3分钟项目介绍

说明：

开题后允许修改题目。

第2阶段：项目开发（3周）

开发过程中建议：

- 使用 Git 管理代码
- 保留阶段性开发记录
- 保存测试截图与运行结果

由于人数较多：

┆ 不安排统一中期答辩。

但最终必须提交：

- Git提交记录
- 阶段性运行截图或测试结果

用于证明开发过程。

第3阶段：项目答辩（第4周）

每组：

- 5分钟演示

需要：

- PPT
- 现场运行演示

若现场条件限制：

可使用提前录制的视频。

第4阶段：最终提交（答辩后1周内）

提交到指定服务器。

十、工作量参考（重要）

项目应体现：

| “一个团队完整完成的小型Linux工程项目”

不要求复杂系统，但需要体现：

- 实际开发过程
 - 团队协作
 - Linux技术使用
-

建议最低工作量

项目通常应包含：

1. 功能模块

至少：

- 2～3个核心功能模块

例如：

- 数据采集
 - 日志分析
 - 自动报告生成
-

2. 代码规模

建议：

- 1000行以上有效代码（不含自动生成代码）
- 或多个Shell/Python脚本协同工作

说明：

并非代码越多越好。

重点在于：

- 是否真正完成功能
 - 是否有合理结构
-

3. Linux技术使用

至少：

- 使用4种以上课程相关技术
-

4. 测试与验证

至少包含：

- 功能测试
 - 运行截图
 - 测试案例
-

5. 开发过程

必须能够体现：

- 阶段开发记录
 - Git提交历史
 - 团队分工
-

十一、最终提交内容

项目最终需要提交：

1. 项目规划书

简化版即可。

2. 项目报告（核心）

参考学校论文格式。

正文建议包括：

1. 项目背景
2. 需求分析
3. 系统设计
4. 功能实现
5. 测试与结果
6. 总结与改进

正文不少于10页（不含代码与附录）。

3. 软件使用说明

包括：

- 如何安装
- 如何运行
- 如何测试

尽量让其他同学也能运行。

4. 项目源代码

要求：

- 提交完整代码
 - 目录结构清晰
 - 有必要注释
-

5. Git提交记录（新增）

必须提交：

- Git仓库
或
- Git log 导出记录

用于展示：

- 开发过程
- 成员参与情况

注意：

| 最终一次性提交大量代码，不属于良好的开发过程。

6. 阶段性开发证据（新增）

至少提供：

- 中期运行截图
- 测试截图
- 开发日志
- 阶段视频

中的任意一种。

7. 项目演示视频

建议：

3~5分钟。

十二、演示要求（重要）

项目答辩必须能够：

| “展示系统真实运行过程”

演示内容建议包括

1. 项目目标

说明：

- 项目解决什么问题
 - 使用了哪些Linux技术
-

2. 系统运行

现场展示：

- 项目启动
- 功能运行
- 输出结果

建议：

不要只展示PPT。

3. 核心功能

重点展示：

- Shell脚本
 - Linux命令调用
 - 自动化流程
 - Agent执行过程
-

4. 测试过程

建议展示：

- 测试输入
 - 系统输出
 - 错误处理
-

演示失败处理

由于环境问题可能导致：

- 软件无法运行
- 网络异常
- 树莓派故障
- 依赖缺失

因此：

项目必须提供：

- 演示视频
- 运行截图
- 使用说明

作为备用证明材料。

若现场无法运行：

将结合：

- 视频
- 代码
- 文档

综合评分。

十三、AI工具使用规范（重要）

允许使用：

- ChatGPT
- Copilot
- Gemini
- Claude
- 各类Agent工具
- 开源代码

但必须满足：

必须做到

1. 标注来源

说明：

- 哪部分参考了网络代码
 - 哪部分使用了AI生成
-

2. 能解释代码

项目答辩时：

应能解释：

- 代码作用
- 系统流程
- 关键实现逻辑

3. 能独立修改

若无法修改或调试：

说明未真正掌握项目内容。

不允许

以下情况可能被视为不合格：

- 完全复制网络项目
- 无法解释代码
- AI生成后未理解
- 无开发过程记录

十四、评分标准

项目	分值
功能完成度	25
Linux技术使用	20
工程规范（Git/结构/注释）	15
文档质量	15
测试与验证	10
创新性与实用性	10
答辩表现	5

十五、课程总成绩构成

平时成绩（40%）

包括：

- 签到
 - 上机实验
 - 平时作业
-

项目成绩（60%）

即课程大作业。

最终要求

课程总成绩达到：

60分

方可通过课程。

十六、优秀项目的特点

优秀项目通常具有：

- 功能完整
 - 自动化程度高
 - Linux特色明显
 - Shell使用合理
 - 文档清晰
 - 测试充分
 - 代码结构清楚
 - 有较好的可维护性
-

十七、常见问题

1. 为什么必须写开发过程？

因为最终代码无法体现：

- 谁完成了工作
 - 是否真正理解
 - 是否存在抄袭
-

2. 为什么需要Git记录？

Git记录能够体现：

- 开发过程
 - 分阶段完成情况
 - 团队协作情况
-

3. AI生成代码是否允许？

允许。

但重点在于：

- 是否理解
 - 是否能修改
 - 是否真正完成项目
-

十八、最后说明

本课程项目强调：

Linux实际应用能力
Shell自动化能力
系统管理能力
工程实践能力

希望同学们能够：

- 学会真正使用 Linux
- 学会利用 Linux 解决实际问题

- 学会使用 AI/Agent 工具提高开发效率
- 建立良好的工程开发习惯